

System Siege — Official Rulebook

Overview

System Siege is a systems-focused engineering competition that tests far more than your ability to write code. Across the event, you will build a real product from scratch, ship it live, defend it against active adversarial testing from other teams, and recover from failures under time pressure.

The event rewards engineering judgment across the full lifecycle — building, breaking, defending, and hardening — not just clean code with zero bugs. Teams that find the most real issues, fix them the fastest, and keep performing until the final whistle will rank highest.

Event Timeline

Phase	Dates & Time	Duration
Phase 1: Build Window	16 Jul , 09:00 AM → 03:00 PM	6 hours
Panel Review and Qualification	16 Jul , 03:00 PM → 16 Jul 2026, 05:00 PM	2 hours
Phase 2: Live Game Window	16 Jul , 05:00 PM → 17 Jul 9:00 AM	16 hour
Phase 3: Final Judgement Round	17 Jul	

Phase 1: Build Window (6 Hours)

The race begins. Your team has 6 hours to design, build, and deploy a fully functional MVP.

What you need to do:

- Design and code your solution from scratch (original work only)
- Deploy it live and make it accessible
- Push everything to GitHub with clear documentation
- Prepare for the storm ahead

What organizers check:

- Does your product actually work?
- Is it accessible and deployable?

- Are your setup instructions clear?
- Do you have a solid foundation to defend?

What you submit before the deadline:

- GitHub repository link — with complete source code, README, and setup instructions
- Deployed application link or a working access method
- Short project description (1–2 sentences on what your product does)
- Domain your project belongs to

Repositories and deployments remain **hidden from other teams** during this phase. No bug reporting is allowed yet.

If a team's deployment is broken at the deadline, organizers may allow a short grace period to fix it before evaluation begins.

Panel Review & Qualification: Once the Build Window ends, a judging panel reviews every submitted MVP and assigns a build score. Based on this score, the **top 40 teams** qualify to move into the Live Game Window. Teams that do not qualify are eliminated at this stage.

Phase 2: Live Game Window (16 Hours)

Repositories and deployments for all 40 qualified teams go public. The leaderboard goes live. From this point, teams play offense and defense simultaneously — a continuous cycle of building, attacking, and defending that runs for 16 straight hours.

What happens:

- **Attack other teams** — test their systems, find real bugs, file GitHub issues with evidence
- **Defend your own** — monitor incoming bug reports, respond fast, patch real vulnerabilities
- **Keep building** — improve your product, add features, harden your system
- **Climb the leaderboard** — every action and decision affects your ranking

What gets scored:

- **Build score** — awarded at the start of the Live Game based on your MVP quality (up to 40 points)
- **Attack points** — earned when your bug reports are validated by the defending team
- **Recovery points** — earned when your fixes are verified by the attacking team
- **Time decay** — unpatched validated bugs reduce your score every minute until fixed

At the end of the Live Game Window, the leaderboard is frozen. The **top 8 teams** advance to the Final Judgment Round.

Phase 3: Final Judgment Round

The top 8 teams present their complete solution before a panel of industry judges. Judges evaluate holistically — not just your ability to attack and defend, but the quality of your solution, your engineering decisions, code clarity, and how well you understood the problem you were solving.

Final rankings are decided by the judges based on their assessment of your project as a complete solution.

What You Can Build

Your project must be deployable, testable, and accessible during the entire event.

Allowed project types:

- Web applications
- APIs with usable interfaces or documentation
- Browser-based tools and utilities
- Dashboards or workflow systems
- Containerized multi-service systems

Not allowed:

- Projects that depend on hardware devices
 - Mobile apps or emulator-based systems
 - Kernel-level OS modifications
 - Non-functional mockups or slides
 - Projects that need manual environment rescue during the event
-

Severity Levels & Staking

All accepted bug reports are classified into one of three severity levels: **SL-1**, **SL-2**, or **SL-3**. Severity is determined by impact on the system, not by how difficult the bug was to discover.

The attacking team selects a claimed severity when submitting the issue link through the portal. The defending team may accept that severity or dispute the report. If the issue

reaches admin review, the organizers may confirm or revise the severity based on actual impact.

SL-1 — Critical

SL-1 applies to issues that severely affect core functionality, trust, security, or data safety. These are failures that would be unacceptable in a real deployment because they compromise the integrity of the system or completely break a critical feature.

Typical SL-1 cases include:

- Unauthorized access to restricted functionality
- Serious data exposure
- Privilege escalation
- Bypass of payment or other critical business logic
- Complete failure of a key feature required for normal system use
- Severe integrity failures that break trust in the product

Examples:

- A non-admin user can access admin-only controls
- Protected records can be viewed without authorization
- A central product flow such as booking, checkout, or login fails completely
- A restricted action can be completed without the required validation

SL-1 should not be used for issues that sound serious but have only minor or limited practical impact.

SL-2 — Moderate

SL-2 applies to issues that do not fully compromise the system, but still cause meaningful correctness, workflow, or reliability problems. These bugs significantly affect normal use and create broken or incorrect outcomes.

Typical SL-2 cases include:

- Broken workflow transitions
- Incorrect calculations or business logic
- Partial feature failure
- Important functionality working only under narrow conditions
- Repeated inconsistent behavior that harms normal usage
- Reliability problems that materially disrupt the product experience

Examples:

- The total amount calculation is wrong under a valid discount combination

- A user can begin a process but cannot complete it because an intermediate step fails
- A dashboard shows inconsistent or stale output due to flawed logic
- A valid flow only works for a very limited set of inputs

SL-2 is appropriate when a bug materially affects usefulness or correctness, but does not rise to the level of a major trust or security failure.

SL-3 — Low

SL-3 applies to lower-impact issues that do not severely break the system, but still reveal weak assumptions, poor engineering clarity, weak documentation, or limited maintainability.

Typical SL-3 cases include:

- Misleading or incomplete documentation
- Weak edge-case handling with limited real impact
- Ambiguous assumptions
- Low-impact inconsistencies in system behavior
- Poor error handling that confuses usage without fully breaking it
- Maintainability or clarity issues that weaken engineering quality

Examples:

- The README claims support for a flow that is not actually available
- A valid but uncommon input causes confusing behavior without breaking the feature entirely
- A setup dependency is missing from documentation
- Error output is misleading even though the product still works

SL-3 should not be used for purely cosmetic preferences unless they meaningfully affect clarity, usability, or understanding.

Severity Interpretation Rule

Severity is based on actual impact, not on:

- How clever the attack was
- How long it took to discover
- How much effort the attacker spent
- How embarrassing the bug looks

If a report overstates severity, the defending team may dispute it, and the organizers may revise the effective severity during review.

Staking System

To raise a bug through the event portal, the attacking team must stake points based on the severity they are claiming. The stake acts as a commitment mechanism and discourages speculative, inflated, or low-quality bug reports.

The exact stake for each severity level will be announced before the start of the Live Game Window.

When the Stake Is Deducted

The stake is deducted immediately when the GitHub issue link and claimed severity are submitted on the event portal. Filing the issue on GitHub alone does not apply the stake; the stake applies only once the issue enters the formal event workflow through the portal.

If the Issue Is Validated

If the issue is accepted by the defending team or ruled valid by the organizers:

- The attacker receives the appropriate bug reward
- The attacker gets the full stake back

The stake is therefore not a fee. It is a temporary lock against the claim while it is under review.

If the Issue Is Ruled Invalid

If the issue is ruled invalid:

- The attacker loses the full stake
- The issue earns no attack reward

Where the event workflow provides compensation to the defending team in invalid-report cases, that compensation is handled through the portal and organizer decision.

Purpose of the Staking System

The staking system exists to prevent teams from flooding the event with weak, careless, or poorly tested bug reports. Because every report carries real point risk:

- Attackers are encouraged to verify issues before filing
- Severity inflation becomes costly
- Defenders face fewer low-quality interruptions

The event rewards quality of attack, not volume of attack.

Responsibility of the Attacking Team

Before staking points on a report, the attacking team is expected to ensure that:

- The issue is real
- The severity claim is reasonable
- The reproduction steps are complete
- The report is within scope
- The report is not misleading

Submitting a weak issue and hoping it survives review is part of the risk the attacker voluntarily takes.

Scoring System

Every team starts the Live Game Window at **100 points**, plus their build score (up to 40 points) added after Phase 1.

Attacking

- File a bug as a GitHub Issue in the target repository, then submit the issue link on the event portal along with your severity claim and stake.
- If the bug is validated → you get your severity reward plus your stake back.
- If the bug is ruled invalid → you lose your stake.

Defending

- An accepted bug bleeds your score continuously over time until it is fixed (see Time Decay below).
- Fixing the bug and getting it verified recovers significant points and stops the bleed.
- The faster you fix, the less damage you take.

Time Decay

The exact decay rate for each severity level will be announced before the start of the Live Game Window.

The moment a defending team submits a fix on the portal, decay pauses immediately while the attacker verifies.

Score Floor

No team's live score can fall below **25 points** during the Live Game Window.

Bug Report Workflow

Filing a bug:

1. Test the target system within ethical boundaries.
2. Create a GitHub Issue in the target repository with a clear title, description, steps to reproduce, observed impact, and evidence (screenshot/video recommended).
3. Submit the GitHub issue link on the event portal, select severity, and confirm your stake.

Defender response (15-minute window): The defending team chooses one of three actions:

1. Accept

- The bug is validated.
- Attacker gets severity reward + stake back.
- Defender loses severity points.
- Time decay begins on the defender to fix the bug.

2. Dispute

- Goes directly to admin review with both sides' statements.
- Admin's decision is final.
- If ruled valid → same outcome as Accept, with a small extra penalty on the defender for disputing a legitimate bug.
- If ruled invalid → attacker loses stake, defender receives half the stake as compensation.

3. Mark as Duplicate

- The defender selects the earlier GitHub issue that this new report duplicates.
- The portal notifies the attacker who filed the newer issue, showing the referenced original issue.
- The attacker then has a response window to **Accept** or **Dispute** the duplicate claim:
 - **Accept** → issue closes, attacker's stake is refunded, no reward and no penalty are applied.
 - **Dispute** → goes to admin review with both issue links and both sides' statements.
 - If admin rules **Duplicate Confirmed** → issue closes, stake refunded, no reward or penalty.
 - If admin rules **Not a Duplicate** → the bug is treated as a valid distinct report and the standard Accept outcome applies in full (attacker reward + stake back, defender penalty, decay begins).
 - **No response within the window** → auto-confirmed as duplicate, issue closes, stake refunded.

No response from defender (15-minute timeout): Auto-accepted, same outcome as Accept.

Fix Workflow

Submitting a fix:

1. Push the fix to your repository.
2. Comment on the same GitHub issue with the root cause and fix explanation, including the relevant commit/PR link.
3. Mark the issue as **Fix Submitted** on the portal. Time decay pauses immediately.

Attacker response (15-minute window):

Verify

- Issue closes.
- Defender receives fix-recovery points.
- Decay stops permanently.
- Attacker receives a small verification bonus.

Reject

- Goes to admin review.
- Decay resumes from the paused point.
- If admin rules the fix valid → issue closes, defender gets fix-recovery points, attacker takes a small penalty for a bad-faith rejection.
- If admin rules the fix invalid → decay continues, defender must submit another fix.

No response (15-minute timeout): Auto-verified, same outcome as Verify.

Rules & Regulations

- **Original work only** — all code, designs, and solutions must be created by your team during the hackathon. No pre-built projects.
- **External libraries are allowed** — open-source frameworks, APIs, and tools are fair game.
- **Deployed and accessible at all times** — your application must be live and reachable throughout the entire event. If it goes down, inform organizers immediately.
- **GitHub is your workspace** — all code pushes, bug discussions, and fixes happen in GitHub. The event portal only tracks issue links, response states, and scores.

- **Bug reports must be valid** — vague, misleading, fabricated, or non-reproducible reports will be disputed and rejected. Provide steps to reproduce and evidence.
 - **Response windows are strict** — defending teams have 15 minutes to accept, dispute, or mark a bug as duplicate. Attacking teams have 15 minutes to verify a submitted fix. Auto-accept/auto-verify applies if there is no response.
 - **No intentional vulnerabilities** — you cannot plant bugs intentionally to farm points or manipulate the game.
 - **Bring your own setup** — all team members must bring laptops, chargers, and any required hardware.
 - **Be present at all checkpoints** — teams must be available at assigned tables during review sessions and interactions with organizers.
-

Disqualification — Unethical Behavior

The following result in immediate disqualification:

- False or fabricated bug reports
- Plagiarism or code theft
- Coordination with other teams to manipulate scores
- Harassment or abuse
- Any misconduct as decided by organizers

Organizers have final say. All decisions on disputes, invalid reports, duplicate rulings, and rule violations are made by the organizing committee. No exceptions.

What You Need to Submit

Before the Build Window ends:

- GitHub repository link (source code, README, setup instructions, known limitations)
- Deployed application link
- Project description (1–2 sentences)
- Domain

During the Live Game Window:

- GitHub issues — bug reports filed against other teams' repositories
- Fixes — comments on GitHub issues explaining root cause and linking to commits
- Fix verification requests — marked on the event portal

